CPRG 304 - Object-Oriented Programming 3

Description
This course explores advanced object-oriented programming concepts, including object-oriented design, integrating software components, recursion and abstract data types (ADT). Introduction to data structures and algorithms for sorting and searching towards the creation of complex programs as a complete client-focused.

3 Credits

Time Guidelines
The standard instructional time for this course is 60 hours.

Prerequisites

- CPRG 211

Course Assessment

Assignments	50%
Learning Activites	50%
Total	100%

Use of Turnitin

The instructor may submit student work in this course to Turnitin's text-matching software program, to help assess the academic integrity of student work in this course. Turnitin results may be considered as one piece of evidence in academic misconduct hearings. Turnitin is an American company that keeps no unencrypted student identity data in the United States unless the students themselves choose to share this information in their submission. Students should limit their sharing of personally identifiable information by not including their names and student ID numbers within the text body of submitted assignments. SAIT has carefully reviewed this company's data management procedures.

Course Learning Outcomes
1. Demonstrate Complexity Analysis

Objectives:

1.1 Describe algorithms.

1.2 Relate efficiency to data structures, data types and algorithms.

1.3 Use big O notation to perform analysis on algorithms.

1.4 Discuss algorithm analysis strategies for problem solutions.

2. Apply sorting techniques.

Objectives:

2.1 Implement linear and binary search algorithms over a collection of data.

2.2 Discuss simple and fast sorting algorithms.

2.3 Demonstrate comparison sorting and non-comparison sorting algorithms.

2.4 Explain time complexity of various sorting algorithms.

3. Examine data structures

Objectives:

3.1 Define data structures.

3.2 Explain the purpose of using data structures for data manipulation.

3.3 Demonstrate common operations on data structures.

3.4 Describe the characteristics of linear data structures.

3.5 Compare physically ordered and logically ordered lists.

3.6 Define Abstract Data Type (ADT)

4. Manipulate lists

Objectives:

4.1 Apply ADT lists in applications.

4.2 Implement the interface of a doubly linked list and its iterators.

4.3 Implement a doubly linked list.

4.4 Compare the performance of various implementations of a list ADT.

5. Manipulate stacks and queues.

Objectives:

5.1 Describe the interface for an ADT stack.

5.2 Demonstrate the implementation process for an ADT stack.

5.3 Apply an ADT queue and stack in applications.

5.4 Interpret ADT queue and stack operations using complexity analysis.

5.5 Describe the uses of the queue and the stack.

6. Manipulate hierarchical abstract data types.

Objectives:

6.1 Define the properties of hierarchical abstract data types.

6.2 Explain binary trees.

6.3 Explain binary search trees (BST).

6.4 Describe the interface for ADT BSTs.

6.5 Demonstrate methods of binary tree traversal.

6.6 Compare binary trees with balanced binary trees.

6.7 Explain algorithms for implementing AVL balanced binary trees.

6.8 Discuss the uses of hierarchical abstract data types.

7. Demonstrate hashing for data storage and retrieval.

Objectives:

7.1 Explain how hashing is used.

7.2 Demonstrate how to create a hash value.

7.3 Demonstrate how to compress a hash value to a table.

7.4 Explain methods for handling collisions.

7.5 Describe the efficiency of hashing algorithms.

SAIT Policies and Procedures:
Students should familiarize themselves with SAIT's Policies and Procedures, including but not limited to those in the Academic Student, Conduct, and Health, Safety, and Environment domains.

Copyright 2015 - 2026, Southern Alberta Institute of Technology (SAIT). All Rights Reserved.

This document and materials herein are protected by applicable intellectual property laws. Unauthorized reproduction and distribution of this publication in whole or part is prohibited.